

```

"""
db.py – SQLite persistence for the Spurs
event-day parking notifier.
"""

import sqlite3
from contextlib import contextmanager
from datetime import datetime

DB_PATH = "spursparking.db"

@contextmanager
def get_conn():
    conn = sqlite3.connect(DB_PATH)
    conn.row_factory = sqlite3.Row
    try:
        yield conn
        conn.commit()
    finally:
        conn.close()

def init_db():
    with get_conn() as conn:
        conn.execute("""
            CREATE TABLE IF NOT EXISTS
events (
                id INTEGER PRIMARY KEY
AUTOINCREMENT,
                event_date TEXT NOT

```

```

NULL,          -- YYYY-MM-DD
                event_name TEXT NOT NULL,
                event_type TEXT NOT
NULL,          -- football / concert / other
                kickoff_time TEXT NOT
NULL,          -- HH:MM, 24hr
                notes TEXT,
                created_at TEXT NOT NULL
            )
        """)
        conn.execute("""
            CREATE TABLE IF NOT EXISTS
subscribers (
                id INTEGER PRIMARY KEY
AUTOINCREMENT,
                email TEXT NOT NULL,
                zone TEXT NOT
NULL,          -- CPZ ref, e.g. NPW,
SA, BC, BGN, BGW, SL, 7S
                lead_time_hours INTEGER NOT
NULL DEFAULT 24,
                reminder_hours_before
INTEGER NOT NULL DEFAULT 3,
                active INTEGER NOT NULL
DEFAULT 1,
                created_at TEXT NOT NULL,
                UNIQUE(email, zone)
            )
        """)
        conn.execute("""
            CREATE TABLE IF NOT EXISTS
scraped_events (
                id INTEGER PRIMARY KEY
AUTOINCREMENT,

```

```

        event_date TEXT NOT
NULL,      -- YYYY-MM-DD
        event_name TEXT NOT NULL,
        source_url TEXT NOT NULL,
        scraped_at TEXT NOT NULL,
        UNIQUE(event_date,
event_name)
    )
    "")
    conn.execute("""
        CREATE TABLE IF NOT EXISTS
notifications_sent (
            id INTEGER PRIMARY KEY
AUTOINCREMENT,
            subscriber_id INTEGER NOT
NULL,
            event_id INTEGER NOT NULL,
            notification_type TEXT NOT
NULL, -- 'advance' or 'reminder'
            sent_at TEXT NOT NULL,
            UNIQUE(subscriber_id,
event_id, notification_type)
        )
    """)

def add_event(event_date, event_name,
event_type, kickoff_time, notes=""):
    with get_conn() as conn:
        conn.execute(
            "INSERT INTO events
(event_date, event_name, event_type,
kickoff_time, notes, created_at) "
            "VALUES (?, ?, ?, ?, ?, ?)",

```

```

        (event_date, event_name,
event_type, kickoff_time, notes,
datetime.utcnow().isoformat()),
        )

def delete_event(event_id):
    with get_conn() as conn:
        conn.execute("DELETE FROM events
WHERE id = ?", (event_id,))

def get_upcoming_events(from_date=None):
    from_date = from_date or
datetime.today().strftime("%Y-%m-%d")
    with get_conn() as conn:
        rows = conn.execute(
            "SELECT * FROM events WHERE
event_date >= ? ORDER BY event_date,
kickoff_time",
            (from_date,),
        ).fetchall()
        return [dict(r) for r in rows]

def add_scraped_event(event_date,
event_name, source_url):
    """Insert a scraped candidate event,
awaiting admin review. Silently
ignores exact duplicates (same date +
name already pending)."""
    with get_conn() as conn:
        conn.execute(
            "INSERT OR IGNORE INTO

```

```

scraped_events (event_date, event_name,
source_url, scraped_at) "
    "VALUES (?, ?, ?, ?)",
    (event_date, event_name,
source_url, datetime.utcnow().isoformat()),
    )

def get_pending_scraped_events():
    with get_conn() as conn:
        rows = conn.execute(
            "SELECT * FROM scraped_events
ORDER BY event_date"
        ).fetchall()
        return [dict(r) for r in rows]

def approve_scraped_event(scraped_id,
kickoff_time, event_type="other",
notes=""):
    """Promote a reviewed scraped event
into the real events table,
    with the admin supplying the
kickoff/start time (not reliably
    available from the scrape) and
confirming the event type."""
    with get_conn() as conn:
        row = conn.execute(
            "SELECT * FROM scraped_events
WHERE id = ?", (scraped_id,)
        ).fetchone()
        if row is None:
            return False
        conn.execute(

```

```

        "INSERT INTO events
(event_date, event_name, event_type,
kickoff_time, notes, created_at) "
        "VALUES (?, ?, ?, ?, ?, ?)",
        (row["event_date"],
row["event_name"], event_type,
kickoff_time,
        notes or f"Source:
{row['source_url']}",
datetime.utcnow().isoformat()),
        )
        conn.execute("DELETE FROM
scraped_events WHERE id = ?",
(scraped_id,))
        return True

def reject_scraped_event(scraped_id):
    with get_conn() as conn:
        conn.execute("DELETE FROM
scraped_events WHERE id = ?",
(scraped_id,))

def add_subscriber(email, zone,
lead_time_hours=24,
reminder_hours_before=3):
    with get_conn() as conn:
        conn.execute(
            "INSERT OR REPLACE INTO
subscribers "
            "(email, zone, lead_time_hours,
reminder_hours_before, active, created_at)
"

```

```
        "VALUES (?, ?, ?, ?, 1, ?)",
        (email, zone, lead_time_hours,
         reminder_hours_before,
         datetime.utcnow().isoformat()),
        )
```

```
def get_active_subscribers(zone=None):
    with get_conn() as conn:
        if zone:
            rows = conn.execute(
                "SELECT * FROM subscribers
WHERE active = 1 AND zone = ?", (zone,)
            ).fetchall()
        else:
            rows = conn.execute("SELECT *
FROM subscribers WHERE active =
1").fetchall()
        return [dict(r) for r in rows]
```

```
def unsubscribe(email, zone):
    with get_conn() as conn:
        conn.execute(
            "UPDATE subscribers SET active
= 0 WHERE email = ? AND zone = ?", (email,
zone)
        )
```

```
def was_notification_sent(subscriber_id,
event_id, notification_type):
    with get_conn() as conn:
        row = conn.execute(
```

```

        "SELECT 1 FROM
notifications_sent WHERE subscriber_id = ?
AND event_id = ? AND notification_type =
?",
        (subscriber_id, event_id,
notification_type),
        ).fetchone()
        return row is not None

def mark_notification_sent(subscriber_id,
event_id, notification_type):
    with get_conn() as conn:
        conn.execute(
            "INSERT OR IGNORE INTO
notifications_sent (subscriber_id,
event_id, notification_type, sent_at) "
            "VALUES (?, ?, ?, ?)",
            (subscriber_id, event_id,
notification_type,
datetime.utcnow().isoformat()),
        )

```